

PLATOLIVES

Version 3.2

User and Technical Manual

A native PLATO terminal for modern macOS

**512 × 512 logical display | 2048 × 2048 Real Plasma rendering | macOS Status Bar
Companion | Native Cocoa & Metal**

Created by Fabio Montarsolo
Release manual | September 2026

About This Manual

This manual describes PlatoLives 3.2 from two complementary perspectives. The first part is intended for users who want to connect to Cyber1 or alternative PLATO services, manage connection profiles, use multiple simultaneous terminal windows, use the PLATO keyboard, select display modes and persistence settings, collect diagnostic information, or automate repeatable sessions. The final part documents the architecture, the 60 FPS zero-copy GPU presentation pipeline, and the Real Plasma renderer in enough detail to support maintenance, calibration, and future platform ports.

Scope: The document reflects the PlatoLives 3.2 design and the consolidated source snapshot. Features listed as future work are intentionally not presented as available.

Acknowledgements

PlatoLives gratefully acknowledges Tom Hunter, Paul Koning, Joe Stanton, Bill Galcher, Steve Zoppi, and Dale Sinder, the authors of the Pterm terminal emulator and Cyber1 contributors. Pterm has preserved over decades a vital body of practical knowledge on PLATO terminal protocols. Specific technical acknowledgement is given to the Pterm source distribution—in particular the implementation in `FrameCanvas.cpp` and `ptermkeytabs.h`—which served as our primary reference specification for decoding esoteric CDC IST-III protocol quirks, escape sequence timings, PTAT keyboard translation matrices, and resolving edge-case bugs on live Cyber1 mainframes. Furthermore, to guarantee absolute, pixel-perfect visual fidelity with classic lessons and games, the canonical M0 (alphanumeric) and M1 (Greek and mathematical symbols) 8×16 font matrices in PlatoLives were directly transcribed and sourced from Paul Koning's Pterm font tables, converting the original 1972 CERL hardware architecture (8 vertical 16-bit column words) into cache-aligned horizontal raster matrices in `plato_font.c`.

Special thanks are also due to Brian Dear (author of 'The Friendly Orange Glow'), the Cyber1 sysops, community beta testers (including Art from group 'cerl'), lesson authors, and archivists who continue to keep PLATO culture accessible. PlatoLives is released strictly under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0) as an open contribution to that shared preservation effort.

Contents

1. Why PlatoLives Exists
 2. Design Principles
 3. Getting Started
 4. How PlatoLives Works
 5. Keyboard and Input Reference
 6. Display Rendering Modes and Persistence
 7. Implemented Features in Version 3.2
 8. Features Still to Be Implemented
 9. Diagnostic Logging
 10. Command File Automation
 11. Performance Strategy
 12. Troubleshooting
- Appendix A. Real Plasma Rendering: Technical Design
 - Appendix B. Historical and Technical References

1. Why PlatoLives Exists

1.1 The practical motivation

Pterm is an important and feature-rich terminal emulator that has enabled the PLATO community to access Cyber1 and related systems for many years. On recent Apple Silicon Macs, building and running software with a long cross-platform history can occasionally require additional setup, Rosetta emulation, or legacy compatibility layers. PlatoLives began as an exploration of a small, lightweight, fully native alternative tailored specifically to the modern macOS toolchain and Apple Silicon hardware.

The project is intended to complement the work that came before it, not to diminish or replace that work. PlatoLives focuses on a compact C11 core, a native Cocoa front end, multi-window and multi-server management, instantaneous builds on modern Macs, and an uncompromising dedication to optical display fidelity, authentic typography, and high performance.

1.2 Project goals

- Run natively on modern Apple Silicon hardware (M1/M2/M3/M4) as well as current Intel Macs (macOS 13+).
- Use modern platform facilities (Core Animation, Metal-ready native Little-Endian BGRA textures, GCD queues) and avoid obsolete X11/GTK layers.
- Multi-window architecture supporting multiple simultaneous sessions on the same or different hosts.
- Keep terminal protocol, font decoders, and rendering logic modular, strictly testable, and isolated.
- Maintain deterministic 60 FPS presentation while keeping CPU consumption minimal (~10% during heavy vector animation).
- Keep the core C11 engine portable enough for planned non-macOS ports (Windows 11).

1.3 Preservation without stagnation

The protocol, terminal behavior, character sets, graphics modes, touch model, and display semantics are the historical contract. The application architecture, build system, memory management, rendering pipeline, and diagnostics are modern.

Design position: Historical fidelity belongs in observable terminal behavior. Modern engineering belongs in the implementation.

2. Design Principles

Principle	Meaning in PlatoLives
Speed	60 FPS hardware presentation, $O(1)$ sliding-window blurs, and near-zero CPU consumption on static screens.
Clean foundation	No requirement for obsolete binary translation (Rosetta), legacy X11 servers, or bloated frameworks.
Portability	Protocol, framebuffer, graphics primitives, canonical fonts, and transport state live in pure, re-entrant C11.
Determinism	The 512 x 512 logical display remains authoritative; optical rendering operates on a strictly decoupled 2048 x 2048 layer.
Verification	Clean-room protocol extraction, focused regression tests, and inspectable, non-breaking changes.
Visibility	Real-time Mach kernel CPU monitoring (F12), diagnostic transport logging, and automated script runners.

3. Getting Started

3.1 Launch behavior

PlatoLives 3.2 starts by loading the designated default connection profile (pre-configured out of the box as Cyber1 on port 8005). It opens an independent, dedicated window with Real Plasma rendering active and applies the profile's saved full-screen, display mode, and persistence settings. The square 512 x 512 PLATO surface is letterboxed within the window while strictly preserving its 1:1 aspect ratio.

3.2 Connecting to Cyber1

1. Launch PlatoLives. The client negotiates the CDC IST-III hardware handshake in the background and presents the authentic Cyber1 sign-on screen: Press **NEXT** to begin.
2. Press **Return** (PLATO NEXT).
3. At the User prompt, enter demo1 and press **Return** (NEXT).
4. At the Group prompt, enter demo and press **Shift+F10** (or **Shift+Ctrl+S** for SHIFT-STOP).
5. At the Password prompt, press **Return** (NEXT) for the public demonstration account. For a personal or author account (e.g. pandar1), enter your assigned password and press **NEXT**.

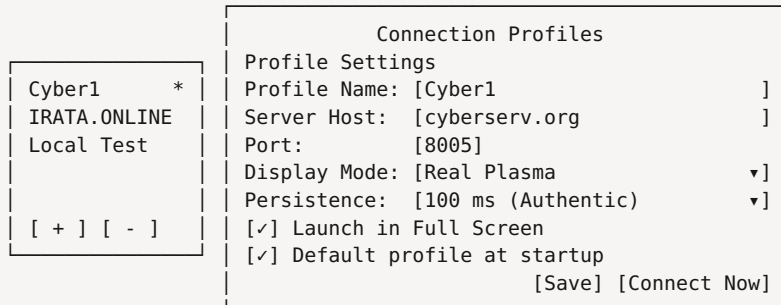
Connection target: Default profile targets cyberserv.org on TCP port 8005.

3.3 Connection Profiles (Cmd+,)

PlatoLives features a profile manager accessible from **Connection → Connection Profiles...** or via Cmd+,:

- **Profile Management:** Create, duplicate, configure, and delete profiles using the [+] and [-] controls.
- **Per-Profile Settings:** Each profile stores Server Host, Port, Profile Name, Display Mode (Real Plasma, Crisp, Split), Persistence Duration (100 ms to 5000 ms), Full Screen at launch, and whether the profile is the system startup default.
- **Instant Connect:** Clicking **Connect Now** closes the active session, clears the screen to clean black, resets terminal state, applies the profile settings, and initiates connection immediately.

- **Dynamic Menu Synchronization (NSMenuDelegate):** The **Connection** drop-down menu is synchronized in real time. Profile additions, edits, and deletions are reflected immediately without restarting the application.
- **Session Lifecycle & Reconnect:** Disconnect (Cmd+Shift+D or profile change) clears the framebuffer to pure black, resets the plasma glow buffers, and preserves the network thread in its listening state, allowing immediate and unlimited reconnections without thread exhaustion.



3.4 Multi-Window Sessions (Cmd+N)

- **New Window (Cmd+N):** Spawns an independent terminal window connected to the default profile.
- **New Window with Profile:** Now available both from the **File** menu and from the Status Bar quick submenu, opening a cascaded terminal window connected instantly to the selected profile.
- **Dynamic Title Bar Metadata (PMD):** The window title is updated in real time by intercepting mainframe metadata traffic (ESC X) and displaying user, group, and slot information. *Format:* PlatoLives: <user>/<group> (<slot>) - <ProfileName> (<host>:<port>) *Example:* PlatoLives: pandar1/brains (4-3) - Cyber1 (cyberserv.org:8005)
- **Session Independence:** Each window runs on its own isolated background network thread, transport socket, terminal RAM, and GPU texture layer. Users can connect simultaneously to different hosts or launch multiple sessions on the same host.

3.5 macOS Status Bar Companion & Retina Dock Icon

- **Status Item in NSStatusBar:** A resident menu-bar icon appears at the top right with a miniature neon-orange plasma glyph.
- **Real-Time Reactivity (menuNeedsUpdate:):** Clicking the status item presents the current connection state, such as Cyber1 (pandar1/brains), Cyber1: Connected, or Status: Ready.
- **Quick Menu: New Window, New Window with Profile ►** (dynamic submenu of all saved profiles, with a star marking the default), **Connect to Default Profile, Quick Connect, Connection Profiles..., Disconnect, About, and Quit.**
- **Official Dock Icon (.icns):** A Retina high-resolution bundle icon compliant with macOS HIG, using a dark ceramic squircle, plasma display, and neon **PLATO LIVES** logo. Startup runtime binding via [NSApp setApplicationIconImage:] in setupDockIcon guarantees immediate Dock presentation even when the application is launched from a terminal or build folder.

4. How PlatoLives Works

4.1 Network and protocol lifecycle

An asynchronous POSIX background thread (`network_worker`) receives incoming stream bytes from the remote host with TCP_NODELAY active and writes them into a thread-safe 64 KB circular ring buffer. A dedicated display timer draining at 60 Hz processes chunks through the PLATO protocol state machine.

On session termination or disconnect, the network worker remains alive in a lightweight sleep state, ready to service subsequent reconnection requests without thread exhaustion or socket hangs. An initial `0x0D` probe byte wakes the Cyber1 ASCII service and initiates the IST-III hardware identification query sequence (`ESC Y`).

4.2 Logical display and canonical typography

The authoritative terminal model is a 512 x 512 1-bit monochrome framebuffer. Coordinate $(0,0)$ resides at the bottom-left, matching original PLATO IV hardware, while raster rows are mapped to top-down memory lines: $ext\{raster_y\} = 511 - y$

PlatoLives embeds canonical CERL 1972 typography transcribed directly from Paul Koning's Pterm font tables:

- **M0 Set:** Authentic alphanumeric 8×16 matrix, including the iconic slashed zero and CERL 1972 proportions, transcribed directly from Pterm and converted to the cache-aligned horizontal C11 raster matrix used by `plato_font.c`.
 - **M1 Set:** Complete 128-glyph alternate set transcribed directly from Pterm. It includes lowercase Greek ($\alpha, \beta, \delta, \lambda, \mu, \pi, \rho, \sigma, \omega$), uppercase Greek (Σ, Δ, Θ), mapped to the corresponding ASCII codes and used by RPGs such as *Moria* and *Oubliette* for monsters and dungeon symbols, together with the assignment arrow ($\Leftarrow$), box glyphs, arrows, and mathematical/set operators.
 - **M2 / M3 Sets:** Programmable RAM character sets stored as 18-bit words and loaded dynamically via `ESC P` and `ESC S`; the implementation has been validated on *CHESS 4.6*.
-

5. Keyboard and Input Reference

PlatoLives provides a unified keyboard engine supporting both historical Pterm conventions and modern macOS shortcuts.

PLATO Key	Primary macOS Shortcut	Pterm / Alternate Shortcuts & Technical Notes
NEXT	Return or Enter	Ctrl+N, Option+N
SHIFT-NEXT	Shift+Return	Shift+Ctrl+N
BACK	Esc or F8	Ctrl+B, Option+B
SHIFT-BACK	Shift+Esc or Shift+F8	Shift+Ctrl+B
STOP	F10 or F4	Ctrl+S, Option+S, Cmd+S (F4 and F10 unified)
SHIFT-STOP	Shift+F10 or Shift+F4	Shift+Ctrl+S, Shift+Option+S
ERASE	Backspace or Delete	Left Arrow, Ctrl+R, Option+R
SHIFT-ERASE	Shift+Backspace	Shift+Delete, Shift+Ctrl+R
HELP	F6 or F1	Ctrl+H, Option+H
SHIFT-HELP	Shift+F6 or Shift+F1	Shift+Ctrl+H
LAB	F7 or F2	Ctrl+L, Option+L
SHIFT-LAB	Shift+F7 or Shift+F2	Shift+Ctrl+L
DATA	F9 or F3	Ctrl+D, Option+D
SHIFT-DATA	Shift+F9 or Shift+F3	Shift+Ctrl+D (Jump to lesson)
EDIT	F5	Ctrl+E, Option+E
SHIFT-EDIT	Shift+F5	Shift+Ctrl+E
COPY	Ctrl+C or Option+C	Sends PLATO key 0x1B (0x03)
SHIFT-COPY	Shift+Ctrl+C	Sends PLATO key 0x3B (0x16)
ANS	Ctrl+A or Ctrl+/ 	Option+A, F2 (Sends PLATO key 0x12 / 0x07)
TERM	Ctrl+T	Shift+Ctrl+A, Option+T (Sends PLATO key 0x32 / 0x14)
SQUARE	Ctrl+Q	Option+Q, F3 (Sends PLATO key 0x1C / 0x7D)
ACCESS	Shift+Ctrl+Q	Shift+Option+Q (Sends PLATO key 0x3C / ESC 0x1D)
SUPER	Up Arrow or Page Up	Ctrl+P
SHIFT-SUPER	Shift+Up Arrow or Shift+PgUp	Shift+Ctrl+P (Dungeon climb stairs)
SUB	Down Arrow or Page Down	Ctrl+Y
SHIFT-SUB	Shift+Down Arrow or Shift+PgDn	Shift+Ctrl+Y (Dungeon descend stairs)
MICRO	Ctrl+M or Option+M	Sends PLATO key 0x14 ('{') to host
FONT	Ctrl+F or Option+F	Sends PLATO key 0x34 ('DEL') to host
TAB	Physical Tab or Right Arrow	Sends ASCII TAB (0x09)
ASSIGN (⇐)	Option + Left Arrow	Dedicated assignment arrow (Ctrl+Left removed to avoid macOS Mission Control conflict)
MULTIPLY (×)	Ctrl+X or Option+X	PLATO multiplication math symbol
DIVIDE (÷)	Ctrl+G or Option+G	PLATO division math symbol

5.1 Fine Grained Touch (FGT) and Coarse Touch

PlatoLives implements native **Fine Grained Touch (FGT)** alongside the legacy *16 times 16* coarse touch grid:

- Clicks within the active terminal area are translated to 512 x 512 logical coordinates.
- The terminal transmits the 6-byte high-resolution FGT sequence (ESC 0x1F <x_lo> <x_hi> <y_lo> <y_hi>) followed immediately by the coarse *16 times 16* keycode (0x100 | (touch_x << 4) | touch_y).
- This ensures full compatibility with both legacy lessons (*Solitaire*, *Freecell*) and precision pointing lessons/games (*Empire* tactical navigation, *Oacidprob* chemistry reactions).

5.2 Clipboard Integration (Paced Paste and Copy Screen)

- **Paced Paste (Cmd+V):** Pasting text from the macOS clipboard executes asynchronously on a dedicated background dispatch queue. Printable characters are sent with a calibrated **250 ms delay** between characters; line breaks (\r\n, \r, \n) are normalized and invoke PLATO NEXT followed by a **500 ms pause**. This prevents host input buffer overruns while maintaining full 60 FPS animation without UI freezing.
- **Cancel Paste (Cmd+.):** Immediately cancels any active asynchronous paste queue running on GCD.
- **Copy Screen (Cmd+C):** Captures the current display state as a clean, high-resolution PNG image directly into the macOS system pasteboard.

6. Display Rendering Modes and Persistence

6.1 Crisp

Direct 1:1 pixel expansion for diagnostic inspections, character set verification, and baseline protocol testing.

6.2 Real Plasma

Authentic optical simulation of the Owens-Illinois gas-discharge neon plasma panel. It models:

- Individual *4 times 4* subpixel cell footprints at *2048 times 2048* resolution.
- Physical neon gas discharge and local glow.
- Wide optical bloom through panel glass.
- Density-dependent cell fusion for solid lines and filled areas.
- Warm unlit glass response.

View	
Crisp	
✓ Real Plasma	
Split: Real Plasma Crisp	
Plasma Persistence ▶	
Show Keyboard Reference	%K
Toggle Full Screen	F11
Show Performance HUD (FPS/CPU)	F12

6.3 Configurable Plasma Persistence

Under **View → Plasma Persistence**, users can select the physical afterglow decay time τ dynamically:

- **100 ms (Authentic Plasma):** Default historical neon-argon PLATO IV persistence.
- **200 ms (Warm Glow):** Softened vector strokes ideal for high-speed action games.
- **500 ms (Medium Persistence):** Typical of P31 green or P40 amber CRT phosphors.
- **1000 ms (Long Persistence / Radar):** One-second luminous trails, in the style of long-persistence P7 radar displays.
- **2000 ms (Ultra Persistence / Phosphor):** A persistent afterglow lasting a full two seconds.
- **5000 ms (5s Extreme / Storage Tube):** Emulation of bistable storage tubes such as the Tektronix 4014 and vector displays of the 1960s.

6.4 Split Mode

Compares Real Plasma (left half) with digital Crisp (right half) rendered simultaneously from identical terminal memory.

7. Implemented Features in Version 3.2

Area	Implemented in 3.2
Transport	Resilient non-blocking TCP socket, automatic reconnection recovery, diagnostic traffic logging (PlatoLives.log), and robust IST-III negotiation.
Profiles	Dynamic connection profiles with real-time NSMenuDelegate synchronization, instant switching, reconnect-safe session lifecycle, and persistent default profile.
Status Bar Companion	NSStatusItem in NSStatusBar with live metadata monitoring and quick-action menu.
App Icon & Dock	Retina .icns bundle icon with runtime [NSApp setApplicationIconImage:] fallback.
Multi-Window	Cascaded multi-window architecture (Cmd+N), per-window independent terminal state, timers, threads, GPU texture layers, and PMD-aware dynamic titles.
Text and graphics	512 x 512 logical framebuffer, Bresenham lines, points, blocks, modulo-512 wrap-around without scrolling, write/erase/rewrite/inverse modes.
Character sets	M0/M1 matrices transcribed directly from Pterm font tables for 100% character-set fidelity; persistent M2/M3 programmable RAM fonts via ESC P and ESC S.
Metadata recognition	PMD (ESC X) decoding with extraction of user, group, and line slot into the window title and F12 HUD.
Pterm-compliant input	26+ key engine including dungeon SHIFT-SUB/SHIFT-SUPER, FONT, MICRO, and non-conflicting ASSIGN mapping.
Clipboard	Paced pasting with 250 ms / 500 ms pauses, asynchronous cancel (Cmd+.), and Copy Screen to system clipboard (Cmd+C).
FGT	Fine Grained Touch plus legacy 16 x 16 coarse touch compatibility.
Audio	Host-requested terminal beep via macOS alert sound (ESC Y 0x7B).
Rendering	60 FPS Real Plasma simulation at 2048 x 2048, Crisp, Split, and 6 selectable persistence levels (100 ms to 5000 ms).
Performance	O(1) sliding-window separable blur, cache-aligned row sweeps, boundary-aware dirty tile tracking, zero-copy native BGRA presentation, and Mach-based CPU telemetry.
Error Handling	Zero-interference failure reporting rendered directly in Real Plasma as ** CONNECTION FAILED **, with no invasive OS popup dialogs.
License	Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0).

8. Features Still to Be Implemented

Future enhancements remain focused on functions not yet implemented in Version 3.2:

Feature	Planned direction
Mouse / trackpad scrolling	Implement scroll-wheel and trackpad scrolling through scrollWheel / ptermSendExt.
Numpad 8-way movement	Add eight-directional numeric-keypad movement for dungeon RPG navigation.
CWS	Implement Character Window Save/Restore for the TERM prompt workflow.
GSW audio	Decode Generalized Sound Writer packets and synthesize musical output.
Windows 11 port	Port the presentation layer to native Win32/Direct3D 11 while retaining the C11 core.

9. Diagnostic Logging

9.1 Enabling the log

Choose **Tools** | *rightarrow* **Enable Diagnostic Log**. The menu updates to *Disable Diagnostic Log* while active. Transport traffic is recorded in `PlatoLives.log` in the application directory.

9.2 What the log captures

- **Session Headers:** Timestamp, PID, direction (TX/RX), byte count, and continuous stream offsets.
- **RAW Dumps:** Hexadecimal and ASCII byte representations before decoding.
- **Protocol Events:** De-duplicated Telnet `0xFF 0xFF` byte pairs, escape sequences, mode changes, and outgoing keycodes.

```
[22:04:03.709] [RX] packet=12 bytes=28 stream=[0x00001A40..0x00001A5B]
RAW  00001A40: 1B 59 70 40 40 1B 63 40 40 1F 5A ...
EVENTS:
00001A40 raw=1B 7bit=1B ESC
00001A49 raw=1F 7bit=1F ALPHA
```

10. Command File Automation

10.1 Purpose

The command-file runner enables automated regression testing, scripted login sequences, and reproducible screenshot generation:

```
PlatoLives --test-script ~/Developer/PlatoLives/test-platolives.txt
```

10.2 Available commands

Command	Behavior
<code>send <text></code>	Send raw ASCII text to the host.
<code>send-env <VAR></code>	Transmit an environment variable value with redacted logging.
<code>key <NAME></code>	Send a named PLATO key (e.g. key NEXT, key SHIFT-STOP).
<code>wait <duration></code>	Asynchronous delay specified in seconds or milliseconds (e.g. wait 2s, wait 500ms).
<code>renderer <mode></code>	Select active renderer (crisp, plasma, real-plasma, split).
<code>screenshot <file></code>	Capture and write internal 2048 x 2048 display to a PNG file.
<code>diagnostic-log <on off></code>	Dynamically toggle transport logging.
<code>quit</code>	Cleanly terminate session.

11. Performance Strategy

11.1 Sliding-Window $O(1)$ Blur Pipeline

In early versions, the optical simulation calculated 4 *times* 4 cell profiles using point-by-point convolution, requiring 68 operations per pixel and consuming over 35% CPU during active screen updates.

PlatoLives 3.2 replaces naive convolution with an exact **$O(1)$ sliding-window running sum**:

- Both the 4-pixel local glow and 12-pixel wide bloom passes execute in constant time per pixel, regardless of kernel radius.
- Vertical blur passes eliminate strided column traversals by maintaining an 8 KB running-sum accumulator (`colSum[2048]`). Because 8 KB occupies only 6% of the 128 KB L1 Data Cache on Apple Silicon, memory operations are streamed horizontally along contiguous cache lines with zero cache misses.

11.2 Boundary-Aware Tile Dilation

The 512 x 512 display is divided into a grid of 16×16 logical tiles (128×128 plasma pixels). A change within a tile only dilates to adjacent tiles if active pixels fall within 3 logical pixels (12 plasma pixels) of the boundary. During dense vector animation (*Asteroids*), active tiles are restricted to only 15–25 tiles per frame, preventing unnecessary full-frame recomputations.

11.3 Zero-Copy Native GPU Presentation

Presentation uses a layer-backed Cocoa view (`wantsLayer = YES`) with an underlying `CALayer`:

- Framebuffers are formatted natively in Little-Endian 32-bit BGRA (`kCGBitmapByteOrder32Little | kCGImageAlphaNoneSkipFirst`) matching the Apple Silicon Metal unified memory texture format (`MTLPixelFormatBGRA8Unorm`).
- Window color spaces are bound directly to `kCGColorSpaceSRGB`.
- This eliminates CPU-side `ColorSync` matrix conversions (`vImage`) and software blitting, dropping presentation overhead to near zero. Total CPU utilization in heavy vector games sits at **~10-12% at 60 FPS**.

11.4 Real-Time Mach Performance HUD (F12)

Pressing **F12** displays the real-time telemetry badge:

- **FPS**: Measured frames delivered per second.
- **PARTIAL / FULL**: Active partial-tile count versus full-screen passes.
- **USER / SLOT**: Live PMD metadata showing the current user, group, and line slot.
- **CPU**: Real process CPU percentage obtained directly from the Mach kernel via `task_threads()` and `THREAD_BASIC_INFO`, with the Apple Silicon Mach tick scaling factor correction ($41.666\times$) applied so the displayed value corresponds to actual process utilization.

12. Troubleshooting

Symptom	Probable cause	Recommended action
**** CONNECTION FAILED ** screen**	Host down, incorrect port, or offline network.	PlatoLives shows the error ** CONNECTION FAILED ** directly in Real Plasma at the center of the screen and updates the window title without blocking the UI.
Screen remains on "Press NEXT to begin"	Normal startup behavior.	Press Return (NEXT) to proceed to the login prompt.
Terminal type queried on login	Handshake sequence desynchronized.	Disconnect (Cmd+Shift+D) and reconnect (Cmd+R). PlatoLives will negotiate the IST-III terminal ID automatically.
Connection fails to down servers (e.g. IRATA.online:8005)	Remote server is down or unavailable.	PlatoLives shows the error ** CONNECTION FAILED ** directly in the plasma at screen center and updates the window title without blocking the UI.
Mission Control switches spaces instead of PLATO Assign arrow	macOS Spaces shortcut conflict.	Use Option + Left Arrow . The old Ctrl + Left Arrow shortcut was removed to avoid conflicts with macOS Spaces.
Dungeon crawler does not descend stairs with SUB	SHIFT-SUB was not sent.	Use Shift+Ctrl+Y or Shift+Down Arrow / Shift+PgDn to send SHIFT-SUB (PLATO code 0x31).
Pasted text drops characters on Cyber1	Host input buffer overrun.	Use Cmd+V (paced paste). Do not paste via external terminal emulators that lack character pacing.
Characters appear in Greek or symbols	Alternate font (M1) active.	Press Ctrl+F (FONT) to toggle back to standard M0 font.
Touch registers in wrong position	Click outside active display.	Ensure clicks are within the letterboxed square plasma panel.
Window closes unexpectedly on Cmd+W	Multi-window shortcut.	Use Cmd+W to close active session; use Cmd+Q to quit application completely.

Appendix A. Real Plasma Rendering: Technical Design

A.1 Historical model

The original PLATO IV display was an AC gas-discharge neon plasma panel invented by Donald Bitzer, H. Gene Slottow, and Robert Willson. The panel possessed inherent bistable physical memory: gas cells fired by alternating sustain voltages remained illuminated indefinitely without external refresh circuitry.

PlatoLives separates logical state from optical presentation. Protocol decoding, vector rasterization, and font blits operate on an authoritative 512×512 1-bit bitboard. An optical pipeline translates this logical state into a continuous 2048×2048 subpixel surface.

A.2 Four-by-four subpixel cell footprint

Each logical cell maps to a 4×4 plasma kernel:

$$\begin{bmatrix} 0.65 & 0.82 & 0.82 & 0.65 \\ 0.82 & 1.00 & 1.00 & 0.82 \\ 0.82 & 1.00 & 1.00 & 0.82 \\ 0.65 & 0.82 & 0.82 & 0.65 \end{bmatrix}$$

Each cell is assigned a stable gain factor between 0.98 and 1.02 at initialization to reproduce subtle, permanent panel variation without animated noise.

A.3 Physical decay and persistence dynamics

When a cell is deactivated in the logical framebuffer, its stored energy decays exponentially:

$$E(t) = E_0 \cdot \exp\left(-\frac{\Delta t}{\tau}\right)$$

The time constant τ is scaled dynamically based on user-selected persistence T_{decay} :
 $\tau = \frac{T_{\text{decay}}}{10.0}$

When $E(t) < 0.0001$, the cell is clamped to zero, allowing the dirty-tile manager to enter an idle noOp state until new data arrives.

A.4 Separable sliding-window blurs

Optical bloom is computed through two separable box-blur passes:

1. **Local Diffusion (Radius 4):** Models light bleed between adjacent cells.
2. **Wide Bloom (Radius 12):** Models light scattering through the glass dielectric and faceplate.

For each line segment $[x_0 \dots x_1]$, the sliding window maintains a running sum $S(x)$:

$$S(x) = S(x-1) + \text{source}[x + R] - \text{source}[x - 1 - R]$$

$$\text{dest}[x] = S(x) \cdot \frac{1}{2R + 1}$$

A.5 Density-dependent fusion and color response

To prevent wide strokes from appearing as disjointed checkerboards, local density blends individual cell cores into solid glowing surfaces:

$$\text{neighborhood} = \text{clamp}\left(\frac{\text{local_glow} - 0.25}{0.40}, 0.0, 1.0\right)$$

$$\text{fused_core} = \text{core} + 0.82 \cdot \text{neighborhood} \cdot (1.0 - \text{core}) \quad (\text{if core} > 0)$$

Color channels are composed into native Little-Endian 32-bit words:

$$R = 0.055 + 0.95 \cdot \text{fused_core} + 0.58 \cdot \text{local_glow} + 0.30 \cdot \text{wide_glow}$$

$$G = 0.016 + 0.31 \cdot \text{fused_core} + 0.060 \cdot \text{local_glow} + 0.020 \cdot \text{wide_glow} + 0.08$$

$$B = 0.004 + 0.018 \cdot \text{fused_core} + 0.003 \cdot \text{local_glow} + 0.010 \cdot \text{hot}$$

Appendix B. Historical and Technical References

- Tom Hunter, Paul Koning, Joe Stanton, Bill Galcher, Steve Zoppi, Dale Sinder, *Pterm: A PLATO Terminal Emulator*, FrameCanvas.cpp and ptermkeytabs.h, Cyber1.org (used as canonical font-table provenance and protocol behavioral benchmark).
- Jack Stifle, *The PLATO IV Architecture*, CERL Report X-20, Computer-based Education Research Laboratory, University of Illinois, May 1971 (revised August 1972).
- Jack Stifle, *The PLATO IV Terminal: Description of Operation*, University of Illinois, revised August 1974.
- Control Data Corporation, *CDC IST-III Terminal Reference Manual*, Publication No. 84002390.
- Brian Dear, *The Friendly Orange Glow: The Untold Story of the PLATO System and the Dawn of Cyberculture*, Pantheon Books, New York, 2017.
- Donald L. Bitzer and H. Gene Slottow, *The Plasma Display Panel—A Digitally Addressable Display with Inherent Memory*, Fall Joint Computer Conference, AFIPS Conference Proceedings, 1966.
- Michael L. McMahan, *The Application of Plasma Panel Technology to Graphic Display Terminals*, Center for Advanced Computation, University of Illinois, January 1972.
- PlatoLives 3.2 source repository, technical architecture and verification snapshot, September 2026.
- Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International Public License (CC BY-NC-SA 4.0), SPDX: CC-BY-NC-SA-4.0.